{"nbformat":4,"nbformat_minor":0,"metadata":{"colab":{"provenance":[{"file_id":"1Wfx6j2bgIPk6sc172Tid4hfgQmqfMgPg","timestamp":1664774525346}],"collapsed_sections":["uBUor5KWp3_Q","FSDbBz7ucm_a","5eh8cI4PeVEb"],"authorship_tag":"ABX9TyNJUEnxpvCg+b/3+7JiXvKy"},"kernelspec":{"name":"python3","display_name":"Python 3"},"language_info":{"name":"python"}},"cells":[{"cell_type":"markdown","source":["## Importing the libraries"],"metadata":{"id":"WT2nf7VJXHsH"}},{"cell_type":"code","execution_count":null,"metadata":{"id":"0kgvCN0n3_1p"},"outputs":[],"source":["import numpy as np\n","import matplotlib.pyplot as plt"]},{"cell_type":"markdown","source":["This assignments has two sections:\n","* Linear Regression\n","* Kernel Regression\n","\n","\n"],"metadata":{"id":"iBY90BqPXO31"}},{"cell_type":"markdown","source":["# Section 1:\n","\n","**Linear Regression**"],"metadata":{"id":"X1nmn2q15Vzn"}},{"cell_type":"markdown","source":["We will use the Boston_housing dataset for the regression problem. Run the below cell to get the following variables:\n","* `Training_data` = Training data matrix of shape (n, d)\n","* `labels` = label vector corresponding to the training data\n","* `test_data` = Test data matrix of shape (n_1, d) where n_1 is the number of examples in test dataset.\n","* `test_labels` = label vector corresponding to the test data\n","\n","Use this dataset for the regression problem."],"metadata":{"id":"aComJ_Fd5dnI"}},{"cell_type":"code","source":["from keras.datasets import boston_housing\n","Train, test = boston_housing.load_data(seed= 111)\n","Training_data, labels = Train[0], Train[1]\n","Test_data, test_labels = test[0], test[1]"],"metadata":{"colab":{"base_uri":"https://localhost:8080/"},"id":"53UCtcH64nsC","executionInfo":{"status":"ok","timestamp":1664774803364,"user_tz":-330,"elapsed":3505,"user":{"displayName":"Nitin Kumar Jha","userId":"04937771533360242308"}},"outputId":"80999439-c3d1-48bb-e213-138c23ee41bd"},"execution_count":null,"outputs":[{"output_type":"stream","name":"stdout","text":["Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/boston_housing.npz\n","57344/57026 [==============================] - 0s 0us/step\n","65536/57026 [==============================] - 0s 0us/step\n"]}]},{"cell_type":"markdown","source":["## Question 1\n","How many examples are there in the training dataset?\n","\n"],"metadata":{"id":"R0Dnj-Qha47p"}},{"cell_type":"code","source":["## Enter your solution here"],"metadata":{"id":"vpLeerN2mGRi"},"execution_count":null,"outputs":[]},{"cell_type":"markdown","source":["## Question 2\n","How many examples are there in the test dataset?\n","\n"],"metadata":{"id":"ChYFPOV5b5jZ"}},{"cell_type":"code","source":["## Enter your solution here"],"metadata":{"id":"0XK7s98bb8PY"},"execution_count":null,"outputs":[]},{"cell_type":"markdown","source":["## Question 3\n","\n","How many features are there in the

dataset?\n","\n"],"metadata":{"id":"ZYLLXlAacL5c"}},{"cell_type":"code","source":["## Enter your solution here"],"metadata":{"id":"rGvr9_jkO7NA"},"execution_count":null,"outputs":[]},{"cell_type":"markdown","source":["Linear regression model for the dataset ${\\mathbb{x}, y}$ is given as\n","$$h_w(\\mathbb{x}) = w_1x^{1}+w_2x^{2}+...+w_dx^{d} = \\mathbb{x}^Tw\n","$$\n","\n","where x^{i} is the i^{th} feature, $\\mathbb{x}$ is the feature matrix of shape (d, n) and $w = [w_1, w_2, ...w_d]^T$ is the weight vector.\n","\n","\n","Notice that above model always pass through the origin but for a given dataset, best fit model need not pass through the origin. To tackle this issue, we add an intercept w_0 in the model and set the corresponding featrue x^{0} to 1. That is \n","\n","$$h_w(\\mathbb{x}) =w_0x^{0}+ w_1x^{1}+w_2x^{2}+...+w_dx^{n} = \\mathbb{x}^Tw\n","$$\n","\n","We call x^{0} the dummy feature and set its value to 1 for each examples. Now w is of shape $(d+1, 1)$ and $\\mathbb{x}$ is of shape $(d+1, n)$ where the first row of $\\mathbb{x}$ has entries as 1.\n"],"metadata":{"id":"KmF4HhMmhx9R"}},{"cell_type":"markdown","source":["## Task \n","\n","Add the dummy feature in the feature matrix `Training_data` and test data matrix `test_data`. We will be using this new feature matrices (after adding te dummy feature) for learning the model.\n","\n","Note: As per your convenience, you can convert the shape of the training dataset to (d, n).\n"],"metadata":{"id":"kN5fsfqfmX_w"}},{"cell_type":"code","source":["## Enter your solution here"],"metadata":{"id":"wic4nhW47fOv"},"execution_count":null,"outputs":[]},{"cell_type":"markdown","source":["## Question 4\n","If the solution of optimization problem is obtained by setting the first derivative of loss function (squared loss) to zero, find the value of $w_0+w_1+...w_d$.\n","\n","\n"],"metadata":{"id":"sK4oWgqCnzgc"}},{"cell_type":"code","source":["## Enter your solution here"],"metadata":{"id":"JORYNRkdOo55"},"execution_count":null,"outputs":[]},{"cell_type":"markdown","source":["## Question 5\n","Find the average of the predictions made by the above model.\n","\n"],"metadata":{"id":"uBUor5KWp3_Q"}},{"cell_type":"code","source":["## Enter your solution here"],"metadata":{"id":"O2vF1aE2Rxfs"},"execution_count":null,"outputs":[]},{"cell_type":"markdown","source":["## Question 6\n","\n","Find the loss for the training data points using the above model. Consider the loss to be defined as\n","\n","$$ \\sqrt{\\dfrac{1}{n}\\sum\\limits_{i=1}^{n} (y_i- \\hat{y}_i)^2}\n","$$\n","\n","Where $\\hat{y}_i$ is the prediction for i^{th} data point.\n","\n"],"metadata":{"id":"FSDbBz7ucm_a"}},{"cell_type":"code","source":["## Enter your solution here"],"metadata":{"id":"F4jRui2VSeDt"},"execution_count":null,"outputs":[]},{"cell_type":"markdown","source":["## Question 7\n","\n","Find the loss for the test data points using the above model. Consider the loss to be

```
defined as\n","\n","$$ \\sqrt{\\dfrac{1}{n}\\sum\\limits_{i=1}^{n} (y_i-
\\hat{y}_i)^2}\n","$$\n","\n","Where $\\hat{y}_i$ is the prediction for
$i^{th}$ data point.
\n","\n"],"metadata":{"id":"5eh8cI4PeVEb"}},{"cell_type":"code","source":[
"## Enter your solution
here"],"metadata":{"id":"vBMCgEIBU6v5"},"execution_count":null,"outputs":[
]},{"cell_type":"markdown","source":["## Question 8\n","Find the weights
using the gradient descent. Use a constant learning rate of $\\eta =
10^{-10}$. Initialize the weight vector as zero vector and update the
weights for 100 iterations. Enter the sum of all the
weights.\n","\n"],"metadata":{"id":"NkeClcplfJLK"}},{"cell_type":"code","s
ource":["## Enter your solution
here"],"metadata":{"id":"QbpyGnfgWEqj"},"execution_count":null,"outputs":[
]},{"cell_type":"markdown","source":["## Question 9\n","\n","Find the loss
for the training data points if the model is learnt using the gradient
descent as in question 8. Consider the loss to be defined as\n","\n","$$
\\sqrt{\\dfrac{1}{n}\\sum\\limits_{i=1}^{n} (y_i-
\\hat{y}_i)^2}\n","$$\n","\n","Where $\\hat{y}_i$ is the prediction for
$i^{th}$ data point.
\n","\n"],"metadata":{"id":"78ApH0oAg96X"}},{"cell_type":"code","source":[
"## Enter your solution
here"],"metadata":{"id":"oI1yIf9N8la7"},"execution_count":null,"outputs":[
]},{"cell_type":"markdown","source":["## Question 10\n","\n","Find the
loss for the test data points if the model is learnt using the gradient
descent as in question 8. Consider the loss to be defined as\n","\n","$$
\\sqrt{\\dfrac{1}{n}\\sum\\limits_{i=1}^{n} (y_i-
\\hat{y}_i)^2}\n","$$\n","\n","Where $\\hat{y}_i$ is the prediction for
$i^{th}$ data point.
\n","\n"],"metadata":{"id":"FA7UKT1Y3PXa"}},{"cell_type":"code","source":[
"## Enter your solution
here"],"metadata":{"id":"tevVzIIj3SZj"},"execution_count":null,"outputs":[
]},{"cell_type":"markdown","source":["## Question 11\n","Find the weights
using the stochastic gradient descent. Use a constant learning rate of
$\\eta = 10^{-8}$. Initialize the weight vector as zero vector and update
the weights for 1000 iterations. . Take the batch size of $⌜\\text{number
of samples}/5⌝ $. For sampling the batch examples in $ith$ iteration, set
seed at $i$. The final weight is the last updated weight. Do not take the
avearge of weights updated in all the iterations. Enter the sum of all the
weights.\n","\n","\n"],"metadata":{"id":"-AoLsBKc31Ul"}},{"cell_type":"cod
e","source":["## Enter your solution
here"],"metadata":{"id":"R4odop9yF9VY"},"execution_count":null,"outputs":[
]},{"cell_type":"markdown","source":["## Question 12\n","\n","Find the
loss for the training data points if the model is learnt using the
stochastic gradient descent as in question 11. Consider the loss to be
defined as\n","\n","$$ \\sqrt{\\dfrac{1}{n}\\sum\\limits_{i=1}^{n} (y_i-
\\hat{y}_i)^2}\n","$$\n","\n","Where $\\hat{y}_i$ is the prediction for
$i^{th}$ data point.
```

\n","\n"],"metadata":{"id":"yPzJLciH4NrV"}},{"cell_type":"code","source":[
"## Enter your solution
here"],"metadata":{"id":"w9usLAPeLNkt"},"execution_count":null,"outputs":[
]},{"cell_type":"markdown","source":["## Question 13\n","\n","Find the
loss for the test data points if the model is learnt using the stochastic
gradient descent as in question 11. Consider the loss to be defined
as\n","\n","$$ \\sqrt{\\dfrac{1}{n}\\sum\\limits_{i=1}^{n} (y_i-
\\hat{y}_i)^2}\n","$$\n","\n","Where $\\hat{y}_i$ is the prediction for
i^{th} data point.
\n"],"metadata":{"id":"rfeamQM94x_N"}},{"cell_type":"code","source":["##
Enter your solution
here"],"metadata":{"id":"oF1xpNH845iH"},"execution_count":null,"outputs":[
]},{"cell_type":"markdown","source":["# Section 2:\n","\n","**kernel
Regression**"],"metadata":{"id":"muMOKLvY5D9O"}},{"cell_type":"markdown","
source":["We will generate the synthetic dataset for the kernel regression
problem. Run the following cell to get the following
variables:\n","\n","`X` = Training data matrix of shape (n, d). In the
given dataset $d = 1$. \n","\n","`y` = label vector corresponding to the
training
dataset"],"metadata":{"id":"pDAKRJua6rCU"}},{"cell_type":"code","source":[
"rng = np.random.default_rng(seed = 101)\n","X = np.arange(-2, 2,
0.01).reshape(-1, 1)\n","y = X**3 + rng.normal(0, 1,
X.shape[0]).reshape(-1,
1)\n"],"metadata":{"id":"_WgICXZSnra0"},"execution_count":null,"outputs":[
]},{"cell_type":"markdown","source":["## Question 14\n","\n","Plot the
scatter plot between feature and the labels. Enter your answer as
0.\n","\n"],"metadata":{"id":"y_-9lyPm8aaj"}},{"cell_type":"code","source"
:["## Enter your solution
here"],"metadata":{"id":"B12Nc2Sv80_s"},"execution_count":null,"outputs":[
]},{"cell_type":"markdown","source":["## Question 15\n","How many examples
are there in the training
dataset?\n","\n"],"metadata":{"id":"E3e-gQHg8z8x"}},{"cell_type":"code","s
ource":["## Enter your solution
here"],"metadata":{"id":"xGeqMCV57ZJB"},"execution_count":null,"outputs":[
]},{"cell_type":"markdown","source":["## Task:\n","\n","Add the dummy
feature in the feature matrix `X`and reshape it to the shape $(d,
n)$."],"metadata":{"id":"uPifeX-K9zuU"}},{"cell_type":"code","source":["##
Enter your solution
here"],"metadata":{"id":"yduBBJQgujfE"},"execution_count":null,"outputs":[
]},{"cell_type":"markdown","source":["## Question 16\n","\n","Our task is
to apply the kernel regression with polynomial kernel of degree 3. We know
that weight vector can be written as\n","\n","$$w =
\\phi(\\mathbb{x})\\alpha$$\n","\n","let us call the vector $\\alpha$ as
coefficient vector. Find the sum of elements in the coefficient
vector.\n","\n"],"metadata":{"id":"HQtBOQua_HQB"}},{"cell_type":"code","so
urce":["## Enter your solution
here"],"metadata":{"id":"CVMDkgkNqCBG"},"execution_count":null,"outputs":[

]},{"cell_type":"markdown","source":["## Question 17\n","\n","Find the sum of the predictions made by the kernel regression model of degree 3.\n","\n"],"metadata":{"id":"Xq0YtsGjA7IK"}},{"cell_type":"code","source":["## Enter your solution here"],"metadata":{"id":"YAqln4GZ05dg"},"execution_count":null,"outputs":[]},{"cell_type":"markdown","source":["## Question 18\n","\n","Find the loss for the training data points. Consider the loss to be defined as\n","\n","$$ \\sqrt{\\dfrac{1}{n}\\sum\\limits_{i=1}^{n} (y_i-\\hat{y}_i)^2}\n","$$\n","\n","Where $\\hat{y}_i$ is the prediction for i^{th} data point.
\n","\n"],"metadata":{"id":"pXRpijIeCcSr"}},{"cell_type":"code","source":["## Enter your solution here"],"metadata":{"id":"8_i2Th-g1ToW"},"execution_count":null,"outputs":[]},{"cell_type":"markdown","source":["### Test dataset\n","\n","run the following cell to get the test data matrix `X_test` and corresponding label vector `y_test`."],"metadata":{"id":"nGpw3zpI65rm"}},{"cell_type":"code","source":["rng = np.random.default_rng(seed = 102)\n","Xnew = np.arange(-2, 2, 0.03)\n","ynew = Xnew**3 + rng.normal(0, 1.5, Xnew.shape[0])\n","X_test = np.column_stack((np.ones(Xnew.shape[0]), Xnew.reshape(-1, 1))).T\n","y_test = ynew.reshape(-1, 1)\n","plt.scatter(Xnew,ynew)"],"metadata":{"colab":{"base_uri":"https://localhost:8080/","height":282},"id":"fLNDYH_B67kN","executionInfo":{"status":"ok","timestamp":1664774902982,"user_tz":-330,"elapsed":722,"user":{"displayName":"Nitin Kumar Jha","userId":"04937771533360242308"}},"outputId":"23f47e9c-4498-4cbc-d1de-816210d576bf"},"execution_count":null,"outputs":[{"output_type":"execute_result","data":{"text/plain":["<matplotlib.collections.PathCollection at 0x7f5ced9eb150>"]},"metadata":{},"execution_count":22},{"output_type":"display_data","data":{"text/plain":["<Figure size 432x288 with 1 Axes>"],"image/png":"iVBORw0KGgoAAAANSUhEUgAAAXIAAAD4CAYAAADxeG0DAAAABHNCS
VQICAgIfAhkiAAAAAlwSFlzAAALEgAACxIB0t1+/AAAADh0RVh0U29mdHdhcmUAbWF0cGxvdGx
pYiB2ZXJzaW9uMy4yLjIsIGh0dHA6Ly9tYXRwbG90bGliLm9yZy+WH4yJAAAgAElEQVR4nO3df
ZBdZX0H8O83mwUuqCw2q8hCTJxabIWR6K2lxqq8FBxRSdEWZ6qVtja1nVphHGwQK+gfTTSOSEc
7nYza0ZHRVKAxLdoADbYjM6FuSDCGF98QZKFlbdn4ki3ZJL/+fce5v+eNZgYRkfHXsroLICIixSiQi4h4ToFcRMRzCuQiIp5TIBcR8dzyOg66YsUKW7V
qVR2HFhHx1u7du39iZpPB150ExcpJXAgXAAOwD8Afmtn/RW2/atUqi0jMjIIIPlI2OuFU
yskpwD8JYCumZ0FYAzA24ruV0RE0nGVI18OoENyNyOAHwfwKIAnABw
ws9uL7ldERNJxkVo5BclAFYDOA3ASSTfrLdepLTJKdnZ2eLHlZERPpcpFYuBPCwmc2amQ2a2QKAW
wG8KriRmW4TTi56rTwK4xbK4FySJwKYB3ABBAHVJEZGRt23PDDbveAiPz83jtIk
Orr74TKxbM+X8OIUDuZndQ/JmAPcCOAxgD4AtRfcrIuKzbXtmcM2t+zC/cAQAMDM3j2u3QcAz
oO5k14rZnadmb3EzM4ys3eY2dMu9isi4qvYiJfL43Hy
m14tQIBcRGVIkJTJ8A1hG4kjIwj2nTXScl1mTZomIDMmbEhncAGbm5mFAaBAnejeGtZt2YtueG
WdlViAXERmSNyUSdgMAgDESQC+ID0L7oJbvKpgdUImyeK0EKyfu2z4V
XERly9cVnojM+tui1zvgYrr74zNjfi7sBlN3wqUAuIjJk3ZopbLzsbExNdEAAUxMa7oaD7aO6LQ73hAHS1fLTUiAXEXEk6gaQFOSLUiAXEalAnlp+Wsq
Ri4h4ToFcRMRzSq2IiKRQ1ZS0eSiQi4hE6J/fPCwYB7d7df14VCAXEQnRX6HJPGwDQrCNQKcFXuqw1pUjV86RzvUyoJmZYWEYMTVYFchFRELtWMVEWB/iFhAbO8WTRWlFrOrQqaRQI8oXD0J+ZLU+aSPAmE+0Hc+P6+7nxNF1V3ss4epHLS+p+IchAsnk9hD5dOS1Ehiua8nPggIqH9jcBwwwtgqr1fkUyg89K7PNaxjG6dZNtdmGN8KBunrjiI1Fs8Gk3BGn8JeLTHhgfwBJTe9JFLNzgB/fHQPxc8c654OUXCWgkgCEl9b7Z5g/SbBr7nG12XG2P02EasTEoagTbjJw/A5gGkQAmzSABVLjFpGUeZWPdFMqmNfAdhPKj2uZBnjhd/6A6CVXkTiJQ/LOWaW8k9ZBSGVn9jCQi68DHGM8kEK
Ri4h4ToFcRMRzSq2IiKRQ1ZS0eSiQi4gkyDr/StVBX6kVEZEWeZfnc654no4fhgFchGRBFl

lY5D/mAk0BOcoLkzSQfJPkAyd90sV8RkSbIMjKzynnIB1zlyG8E8K9m9laSxwE40dF+RURiVZG
PvvriM5eMzASAg4cO44Pb9uGuB2ePHX/ixHE8dXBhyT7KmId8oHAgJ3kygNcAuAIAzOwQgENF9
ysikqSqdTEH+7p++37MzT8TpJ86uIAv7nr02Pczc/MYX0aMjxELR56Z79DlcPwwLmrkqwHMAvg
Hki8DsBvAe83sF8MbkVwPYD0ArFy50sFhRWQUJa3CM8hHZ1nNJ6omH9ymP7V4rIWjhonOOE46f
nllvVZcBPLlAF4O4D1mdg/JGwFsAPDXwxuZ2RYAWwCg2+0uXTpDRCRBsAYetgoPkJyPTlOTD9s
mrQPzC9h73UWpty/KRWPnYwAeM7N7+t/fjF5gFxFxKmoVnqCkfHSaniVpj5Xn+K4VDuRm9l8Af
kxykAC6AMD9RfcrIhKUpudHmnx0mp4leXuZlJ0PD+OqH/l7ANxE8tsAzgHwN472KyJyTFRNd4z
MtAhEmu6EUdtMdMYXLTrx9nNXZl6EwjUn3Q/NbC+Arot9iYhECesG2Bkfyxw8o/YzXJOO2ub6N
7+0MXOsDGiuFRHxhqsFGtLsp+zFIFyiRbT6lqnb7dr09HTlxxURcaWO2RBJ7jazJdkP1chFRDK
qaiBSWpo0S0QkozomxoqjQC4iklEdE2PFUWpFRLxU54o9p010Qkd6Vj0QaEA1chHxTh2LNwy7+
uIz0RkfW/RaHQOBBhTIRcQ7deeo162ZwsbLzq59INCAUisi4p0m5KjXrZlqTJ9y1chFxDtZVuw
ZBQrkIuKdpuWo66bUioh4x6fh81VQIBcRL5WRo66zS2MRCuQiImjesPsslCMXEUH9XRqLcBbIS
Y6R3EPyX1ztU0SkKk3o0piXyxr5ewE84HB/IiKV8blLo5NATvJ0AJcA+IyL/YmIVM3nLo2uGjs
/CeD9AJ7taH8iIpXyuUtj4UBO8o0AnjSz3SRfF7PdegDrAWDlypVFDysi4lyTht1n4SK1shbAm
0n+CMCXAZxP8ovBjcxsi5l1zaw7OTnp4LAiIgI4CORmdo2ZnW5mqwC8DcBOM3t74ZKJiEgq6kc
uIuI5pyM7zewbAL7hcp8iIhJPQ/RFpLF8nfukagrkItJIZc190sabgwK5iDRS3NwnaQNvMGif9
5JJ3LJ7xsuJseIokItIraJqyEXnPgmr0d+061FYYLusN4cmUiAXkdrEpU9Om+hgJiRop537JKx
GHwziAz5MjBVH3Q9FpDZx6ZOic59kCc7LSKzecBvWbtqJbXtmUv9eUyiQi0ht4tIn69ZMYeNlZ
2NqogMCmJroYONlZ6dOgUTV3Bny2hEzGJ55IvAtmCu1IiK1SUqfFJn75OqLz1yUtgF6Nfq3vGI
Kdz04i8fn5rGMxBFbnHDxMWeuQC4itYkKti6mjk0zm+HqDbeF/q5vOXMFcmmlNvYV9lHS51DW1
LHB495w+Tmh+yzaoNoUCuTSOj4votsmaT8H11PHZvn8y3wiqJIaO6V1fF5Et03q+hyyHLdog2p
TqEYurePzIrpFNSmlVNfnkPW4vi4mMUw1cmkdnxfRLWKQUpiZm29EV7q6PodR/PwVyKV1fF5Et
4impZSSPodte2awdtNO5wNxRvHzV2pFWsfnRXSLaFpKKe5zKLNBehQ/f5pFzT6QcgfkGQC+AOD
56E1lsMXMboz7nW63a9PT04WOKyKLrd20M7Qr3dREB3dvOP/Y903Io6ctqyxGcreZdYOvu6iRH
wbwPjO7l+SzAewmeYeZ3e9g3yKt5yqwpulK15SumVFPCTNz81i7aefI1KRdKRzIzewJAE/0v/4
ZyQcATAFQIBdJ4DKwpkkpuJjj24WogTgEjr0+fC6AYqmSJjyFlKlwamXRzshVAP4DwFlm9tPAz
9YDWA8AK1eufMUjjzzi7LgivopKMQC9NIPrgLN6w22hU7kSwMObLnF2nCTBG9igDGFlm+iM4+n
DR5c8aaTt7x12rCy/3yRRqRVnvVZIPgvALQCuDAZxADCzLWbWNbPu5OSkq8OKeC2uIbKM7oNN6
ZoXNhAnqko5N7+QujdOWE+YpvXmKYOTXiskx9EL4jeZ2a0u9inptf2xsSp1nMeoFMOA67RHk4a
kBwfixD2dhAneBKPSVMEgHvX7PitcIydJAJ8F8ICZfaJ4kUZPkf60TRsEUiWX/ZDrOo9hfZ6DZ
ubmnfW1bvKQ9Kj+36ecOB66ffApIqrmPcawGcjbNUDIRY18LYB3ANhHcm//tQ+Y2dcc7Lv1ijZ
2NaXxqmque1/UdR6HGyjjaqPDN5fh38t7TBfvyfUTTFRjLYBUTxFRNewjZuiMjzXiKaQsLnqtf
BPhi25ICkUDSNMGgVTFdeCt8zwOAmtYo1xQ1TfpqGBdVjfGuJtM0k0jKk01aDRuc/pRIztrVjS
AtGU+5axcB94mnMdgjbSOhYKHA/fJnXH84tBhLBzplWQ4WJf9BBN2A0kaKBSX/2/DxFhxFMhrV
jSANKnxqsrGQheBNxi0xsd4LGgB9ZzH4YAT1fhX1s0lWMuem19Yss0gWJf5BJO3tj+KQ/MHNGl
WzYpO8DPceAUAY+Sxi63KBs+qGwuLnrdgeefmFwADTjlxvDGNgFVP/hRWyw4zCJJhXNxkinQXX
LdmCndvOB8Pb7oEd284fySCOKAaee1c1CIG21Y99Hq4Rlv1IrZFz1tYsFg4ajjxuOXY86GLnJc
3j6prmFnSeWU+CY5qu08RCuQN4CJ/V3Wvi+DjbzCID5R58aU5b1HpHl+CRd6/jTxprqQ+7cDin
DNQzk2mCe0VvlEgb4mqA1Pax/A6L764XGubg0XeHHNYLXt8GfGsE5Zj7uDCkmBdVgNik9p9fKF
A3hJVB6Y0N4i6L764p5QiwaLpI2nzPp3lrWVX1Z+8See4aRTIW6LqWkzUjWOMxFGzRlx8cU8pw
YE4w43EQHTNNam2W1aQz7LfIk9nWdNVcV0UmzBoaVQokLdE1bWYqBtHnT09gsFu4sRxPHVwaRe
6wVNKnkbipB4VUfsa/G6ezyZrqqTMp7O0XRSv375fNeoKOZ3GNi2tENQOTUoxhI2KHF9GgFjSN
3z4ZpN1pZq4aWCjAmjRaVizlrHMaVuzTmzl+vijrswVgmRENenxN6o74URnHCcdvzzyZpM1DRF
X2436nbiBNWVMwxD2dHbeSyaxecdDuGrr3kI33byN56Mw/0+dFMglsybVxAeiAsyB+QXsvS66X
3jWNERcW0TSxFdpy1y0jMDim6zLeVHSdFGM0rSunW2ikZ0SKzhV7Ae37WvktLl5RxpmHT0ZNw1
s0WlYXZUxKCqvf/32/ZmnAQ4ry/gyLhoRW/T9SnaqkUuksJrcTbseXZIjbsJjc95eO3kaiaNSS
kWnYXVZxmFxKZ9B2sflfCZROXr1Ay+PGjslUpaGrarXfAzTxJTPQJ1ly/I5RjWgZhX3fpv8OTV

dqY2dJF8P4EYAYwA+Y2abXOxX6pUlp5n02FzFxdukxtc007BWFdDCnlaiuMpjR30WZc1jPuoKB
3KSYwA+DeC3ATwG4Fskt5vZ/UX3LfHSBIK8wWLbnpnQibCApaudJz02l33xlnke8pYn6f1WGdD
C0iEHDx2O7WNfllFd0apsLmrkrwTwfTP7IQCQ/DKASwEokJeozGAx+L2wIN4ZH8NbXjGFux6cT
R0Uy7x4mxY0gXTv1+U5SXOTCtaQy8pjJ5XFl8nKfOMikE8B+PHQ948B+A0H+5UYLoJF1EUXNSH
WGJlrUEeZF29ZQbNIDT7qfc3MzWPbnhmnsy82aRGGNGVp82Rldaqs+yHJ9SSnSU7Pzs5WddjWS
hMI4gLKOR++HVdu3RvajTDq946a5brQy1yEoMh5iHq96CIZce9rsB9X58TFIgw3XH4OAOCqrXt
Td0PMW5aqF8sYFS4C+QyAM4a+P73/2iJmtsXMumbWnZycdHDY0ZYmEERtQ8SPNnQdeMu8eIuch
6jXiwRHIPz9Bvfj6pwUrdm7XNkpTVni+uFLfi4C+bcAvJjkapLHAXgbgO0O9isx0gSCsG2CDZV
Bj8/NOw+8ZV68ec9D3PspGhwH7zfKYPZFF+ek6E236E0rT1kGTwKjthxbmQrnyM3sMMm/ALADv
e6HnzOz/YVLJrHS5DjDtknqT7yMBABsvOxs53NMl3HB5j0Pce/HRR530NYQtx8X56To9MUu2y+
0IER9NCCoRnUMjEgzOGTUZ6oL69EB9BZmvu5NL809/SxQzrkt8neUdWbFMssiyaIGBCmQ16Tsi
zzqgooKUkGuRvj5atueGVy/ff+StoSsn1HTA5sPNxt5hqaxbZi6+1YPLqqo2/io9+sdpEaCgTz
rZ9Sk0aZhqliQRKM5y6dAXpI6B0Yk3SSGg0vUo7X69ab/jHyvbZZ9s9FozvJpGtsSpOnSFRUol
5GZphUNk+UmUWe/3uAUuXVPhRuUpheGy+57baXRnOVTIC9B3oERAHDErHBAyNIlrayugUlB2oc
AmOYm57L7XluVOSBMepRaKUHagRHAM7nJsAmq8j5+Zu0G5vrROm2OPs3jdp1pizT5Y9U2k6lbY
vkUyEuQth/ycABdveG20H3lCQhVNGDFSROk46YPWL3htmPrTN6ye6bWRrKkm5zmDklW99/jKFA
gL0GeGojrgFBnb4k0tdS4wUmDVEtTVyMa1vTaZlMaYpvee8d3ypGXIE/euU2TCaXJicbNRzLgQ
9fIJs8d4kM7hLihGnlJstZA2vT4mVRLHdQS5xeOYCxi8Yo4TUtbVF3bTFvLrnrOc6mPAnmDNPn
xM8uFHHdTCjaEHjFDZ3wMJ4wvC12xJutqRG2XZXBN3XOeS3UUyCVRngs56qYUVUs8fvkydMbHl
tTis65G1HZZatmu2l00oKf5FMglkcsLOao2eGB+ATdcfo4e3xNkHezloiFWXSybT4F8RJSxdFm
eCzmqlriMxFVb9+K0iQ5uuPycUgO4z/neLLVsV+0u6mLZfArkDeUy2BTNcbq8kMNqiQCONXhGl
c3V+fA931vHYK+md7EUdT9spLBuY1d/5T6s+cjtueYlKWPpsrwXcrC73lh/IYu4srnsRuf7kPo
6ujs2uYul9BSqkZPcDOBNAA4B+AGAPzSzORcFq4LrR2xX+wsLNgtH7Vivjqy1SBdLlw3KlfW9R
Z2TLCNaq8jR+5TvHZy/wbm9auveY+uAlhVcm9yjSoqnVu4AcE1/ubePArgGwF8VL1b5wh6xr9y
6Fx/+5/2ZVoGJ299wsM0S5NMElSyBzNXSZa7PSVzZTu6MY+2mnc7nTG9Lvtf3FJG4VSi1Yma3m
9nh/re7AJxevEjVCKvlAcBTBxdyPbbH1RqzpgbSBpW0gayuUaN5Z4EcX0b84tDhY+crSt4cfRt
G0PqeIhK3XObI/wjA16N+SHI9yWmS07Ozsw4Pm09cEIy7IKKmZ417ZM960aUZvg6kD2R15TjTz
gIZLNuzTliOhSPxoz2J3g0xa3tBW/K9bUgRiTuJqRWSdwI4NeRH15rZV/vbXAvgMICbovZjZls
AbAF6a3bmKq1DSSvKh10QcY+zcY/sWS+6YE765M44fnHo8KLglrUWWUeOM88skEB03nxgeLRnn
pSCD/nepFRcW1JE4kZijdzMLjSzs0L+DYL4FQDeCOD3rY6VnHNKqvWGXRBxNeu4R/Y8E+uvWzO
Fuzecj4c3XYK9112EzW99mXe1yLxpjKjzMjXRwdREJ3JGxLZIk4prS4pI3Cjaa+X1AN4P4LVmd
tBNkaoxCIJRK6WHXRBxNeuknh1F++H6UIsMytvbJa7f8lVb94b+Th0phbIGFqXppdOmSdakuKK
9Vj4F4HgAd7DXH3iXmb27cKkqEuzGlXRBJD3ORgXbUb7o8tyA4s7X5h0PNSKlUGavkbSpOB9v7
lKOQoHczH7ZVUHqlPaCKDLCTRddNlHnqymjDMucSKoJ+W+fpzEYRRrZmUFbejz4rCmfQZm9Rur
Of2tBCv9orpWMVLOuXxM+gyK15qTabt2pOE1b6x8FcpEc8qZ40ubW67xZqY+6f5RaEckhb4rHh
xGZebrLSr1UIxfJKU+t2YfablMalCU91chFKuRDbbcpDcqSnmrkIhXypbbbhAZlSU+BvGTqj9s
Orj7HunukSDspkJdIc0a3g+vPUbVdcU058hL50ENBkulzlKZTjbxEPvRQcKHt6aNR+RzFX6qRl
8iHHgpFjcJw7lH4HMVvCuQlqnvOjCqMQtphFD5H8ZtSKyUahR4Ko5B2GIXPUfymQF6ytvdQaMK
Uq1Vo++cofnOSWiH5PpJGcoWL/ZUhatFkKUZpB5H6Fa6RkzwDwEUAHi1enHKoP3d5lHYQqZ+L1
MoN6K3b+VUH+yqF5lcul9IOIvUqlFoheSmAGTO7z1F5SjEKDXIiMroSa+Qk7wRwasiPrgXwAfT
SKolIrgewHgBWrlyZoYjFldUg1/aBMCLih8QauZldaGZnBf8B+CGA1QDuI/kjAKcDuJdkWNCHm
W0xs66ZdScnJ12+h0RlNMiNwkAYEfFD7hy5me0D8LzB9/1g3jWznzgoV6Q8teAyGuTqzLvrSUB
EhnnVj7xI75OiDXLB4BmWqgHS5d2LBGL1wBGRIGdD9M1sVdm18bqGg4elURixbVLevWhKpsxzo
L72In7yqkZeV++TsOBpANj/f9jBQ4exbc9MZO04a0rG5ZNAHNX0Rfzl1aRZdc1CFxUkDcBEZ3z
Ra08dXIitYWe5Gbl8EkgyCpNfibSVV4G8yuHgw2mGZQwPn1MTHZx0/NKHmrgAmOVmFPckMMzFO
VBfexF/eRXIq1rdO1gTPmLBBMozwTNrAMxyM4p7EnB9DjTntoi/vMqRA+l7nxTpGRJWEwaAMRJ
HzRbtb/OOhzINNsrSFTIqJz410cHdG85P9V7S8mV1dxFZyrtAnkbRhruomvARM3zy8nMW7SNPA

Ex7M6oyuGryKxF/tTKQFx2sE9c7JHhDKDMAVh1cNfmViJ9oIfnfsnW7XZueni5t/6s33LakWyD
QayR8eNMlib8frNEHlZHaEBFJQnK3mXWDr3vV2JlW0Ya7QaNqFPXkEJEmaWUgd9FNcd2aKUypJ
4eIeKCVgdxVN0UtYyYiPmhlYyfgpuFOPTlExAetDeSuqCeHiDRdK1MrIiKjRIFcRMRzXqdWtFK
OiIiDGjnJ95B8kOR+kh9zUag0tGamiEhPoRo5yfMAXArgZWb2NMnnJf2OK2mH4avWLiJtVzS18
mcANpnZ0wBgZk8WL1I6aaaP1ao3IjIKiqZWfgXAb5G8h+S/k/z1qA1Jric5TXJ6dna24GHTDcP
XqjciMgoSAznJO0l+J+TfpejV6J8L4FwAVwP4RzJ8OR0z22JmXTPrTk5OFi54mlGXWvVGREZBY
mrFzC6M+hnJPwNwq/WmUPxPkkcBrABQvMqdIM2oy6jpaDVXioi0SdEc+TYA5wG4i+SvADgOwE8
KlyqlpFGXWvVGREZB0UD+OQCfI/kdAIcAvNPqmOA8guZKEZFRUCiQm9khAG93VJZSaK4UEWk7D
dEXEfGc10P0XdPgIRHxkQJ5nwYPiYivlFrp0+AhEfGVNzXystMeGjwkIr7yokZexUyHaYb8i4g
0kReBvIq0hxZaFhFfeZFaqSLtocFDIuIrLwJ5VXOmaPCQiPjIi9SK0hxZaFhFfeZFaqSLtocFDIuIrLwJ5VXOmaPCQiPjIi9SK0h4iItG8qJGnTXtoQI+Ij
CIvAjmQnPbQgB4RGVVepFbS0IAeERlVrQnkGtAjIqOqNYFcA3pEZFS1JpCrZ4uIjKpCgZzkOSR
3kdxLcprkK10VLKt1a6aw8bKzMTXRAQFMTXSw8bKz1dApIq1XtNfKxwB82My+TvIN/e9fV7hUO
WlAj4iMoqKpFQPwnP7XJwN4vOD+REQko6I18isB7CD5cfRuCq+K2pDkegDrAWDlypUFDysiIgO
JgZzknQBODfnRtQAuAHCVmd1C8vcAfBbAhWH7MbMtALYfbtdwlLkijP0WkbiWP6aSPABgw
syMJAEcMLPnJP2UEcMLPnJP1et9u16enp3MfNKzj6E+j1bFGjqIj4gORuM+sGXy+aI38cwGv7X58P4HsF91c
qjf4UkTYqmiP/EwA3klwO4P/Qz4FXLW26RKM/RaSNCgVyM/smgFc4KksuWSbLqmpecxGRG/s
jNLukSjP0WkjbyZxjZKlnSJlnMTkTbyPpBnTZdo9KeItI33qRWlS0RkRGnfeBHFC
6RERGm/epFRGRUadALiLiOQVyERHPKZCLiHhOgVxExHOFprHNfVByFsAjOX99BYCfOCyOKypXd
k0tm8qVjcqVTZFyvdDMJoMv1hLIiyA5FTZ6qW4qVzYqVzYqVzbOy+VdjlxERHBbzsUYuIioiI5xTIRUQ852M
g31J3ASKoXNk1tWwqVzYqVzbOy+VdjlxERBbzsUYuIiJDFMhFRDzX+EBOcjPJB0l+m+Q/kZyI2
O71JB8i+X2SGyoo1++S3E/yKMnIrkQkf0RyH8m9JKcbVK5Kz1f/mM8leQfJ7/X/PyViuyP987W
X5PaSyhL7/kkeT3Jr/+f3kFxVRjlylu0KkrND5+hdFZTpcySfJPmdiJ+T5N/2y/xtki8i8vu0wpy
/U6kgeGztWHKirXGSTvInl//3p8b8g27s6ZmTX6H4CLACzvf/1RAB8N2WYMwA/AvAjAcQDuA/B
rJZfrVwGcCeAbAbALox2/0IwIzk1diueo4X/3jfgzDAy/tfPxv
Ad0M+R2fnrPE1cjO73cwO97/dBeD0kM1eDDxMzsE4MsALXA+Y2dIAuWphzZk+Y2b3
wF8vv/15wGsq+CYYdK8/+Gy3gzgApJsSNkqZ2b/AeB/Yza5FMAXrGcXgAmSL2hAuWphZk+Y2b3
9r38G4AEAwUUTnJ2zxgfygyD9C7w4WNAXgx0PfP4alJ60uBuB2krtJrq+7MH11na/nm9kT/a//C
8DzI7Y7geQ0yV0kywj2aVwzHKRsAvKX/OH4zyTMqKFeSJl+Dv0nyPyPpJfJ/n
Sqg/eT8utAXBP4EfOzlkjVggieSeAU0N+dK2Z2fbW/zbUADgO4qUnlSuHVZjZD8nkA7iD5Y
Xe5ShFXtuFvzMxIRvV9fWH/nL0IwNdDNTu1S+8zsB67L6rF/BvAlM3ua5J+jV7t/fwt1laqp70ft7+jn
JNwDYBuDFVR2c5LMA3ALgSjP7aVAgSP7aVnHaUQgN7ML435O8p0ALgZwgfWrCgGPATij6/vT+68
dP//0mS/4Teo3OhQ6g7XOCLxQmHyhwNnZ+8EBa+CyCjqCVQK4B8YQOb8YQNDKHlGAMVAgIWX
lAE4G8D8Oy5C7bGHY2XI7Potmdf2ULfS/qagmGA6eZvY1Y1kn9HcoeAXsZK
1MrJF8P4P0A3mNRcQxmDwym+M6+xAF5NcTfI49BqnSunkOJk7xWQgpxkPAnGr
EsOTpgeQpJI/vf70CwFoA9zsuR5r3vw3AIQAr0CWhCS/Pxn5pb//foCfBaQC+FmgB/i5
6NbdrKyjX76CX03oawH8D2Bes3Fo9D+7r/9vflHLVcb76x/wlAP8G4HsA7gTw3P62EwB+iH9X/9K
gD7+udsH4A/LqksS94/gI+gV2EAgBMAfKX/9/9efAF5UxTlWwCeALD
Q//v6YwDvBvDu/s8J4FP9uRM9DTEuisv1F0POr+uisvn1F0PnausZEVwAAABGSURBVBeAV1VUrlej1z727aHY9
YayzpmG6IuIeK7xqRUREYmnQC4i4j4jkFchERzymQ4h4ToFcRMRzCuQiIp5TIBcR8dz/AyoDpiv
GA/ijAAAAAElFTkSuQmCC\n"},"metadata":{"needs_background":"light"}}]},{"cel
l_type":"markdown","source":["## Question 19\n","\n","Find the loss for
the test data points. Consider the loss to be defined as\n","\n","$$
\\sqrt{\\dfrac{1}{n}\\sum\\limits_{i=1}^{n} (y_i-
\\hat{y}_i)^2}\n","$$\n","\n","Where $\\hat{y}_i$ is the prediction for
i^{th} data point.

\n","\n"],"metadata":{"id":"5vlunIBzDI1_"}},{"cell_type":"code","source":[
"## Enter your solution
here"],"metadata":{"id":"_RxSPslY7joK"},"execution_count":null,"outputs":[
]}]}